

Topic 1

Introduction to Numpy

Yehonatan Bar Moshe
Bamoshe2@mail.tau.ac.il

What is NumPy?

- NumPy stands for "**Numerical Python**"
- It is a **library** for the **Python** programming language.
- Provides support for large, multi-dimensional **arrays** and **matrices**.
- Offers a collection of **mathematical functions** to operate on these arrays efficiently.

Key Features of NumPy

- **N-Dimensional Array:** Powerful N-dimensional array object.
- **Universal Functions (ufuncs):** Perform element-wise operations.
- **Broadcasting:** Allows for operations on arrays of different shapes.

N-Dimensional Array

Definition:

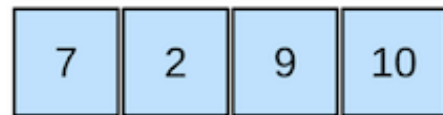
- Nd-arrays are the core data structure of NumPy.
- They are **multi-dimensional, homogeneous** arrays of fixed-size items.

Attributes:

- **Shape:** Tuple indicating the size of each dimension.
- **Size:** Total number of elements in the array.
- **Dimensions (ndim):** Number of dimensions (axes) of the array.
- **Data Type (dtype):** Type of elements stored in the array.

N-Dimensional Array

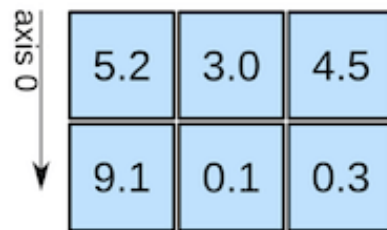
1D array



axis 0 →

shape: (4,)

2D array

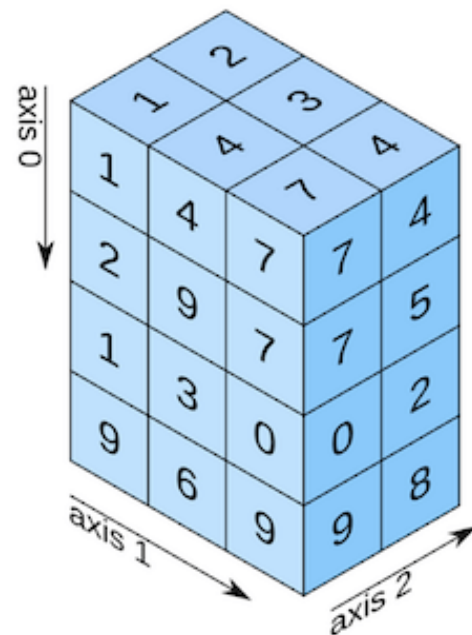


axis 0 ↓

axis 1 →

shape: (2, 3)

3D array



shape: (4, 3, 2)

Universal Functions

Definition:

- Universal functions (ufuncs), are functions that operate element-wise on ND-arrays.

Purpose:

- Provide a fast and efficient way to perform element-wise operations.
- Support broadcasting, type casting, and several other standard features.

NP-Array vs. Lists

NumPy arrays and lists can both be used to store collections of items, but they have significant differences in terms of functionality, performance, and use cases

1. Data Type Consistency

- **NumPy Array:** All elements in a NumPy array must be of the same data type. This uniformity allows for more efficient memory usage and faster operations.
- **List:** A Python list can contain elements of different data types. This flexibility comes at the cost of performance and memory efficiency.

2. Performance

- **NumPy Array:** NumPy arrays are implemented in C, which makes operations on them much faster than operations on lists, especially for numerical computations.
- **List:** Lists are built-in Python data structures and are generally slower for numerical operations because they are not optimized for these tasks.

3. Functionality and Operations

- **NumPy Array:** NumPy provides a wide range of mathematical functions and operations that can be performed directly on arrays, such as element-wise addition, multiplication, and more complex operations like matrix multiplication, Fourier transforms, etc.
- **List:** Lists do not support element-wise operations directly. You would need to use loops or list comprehensions, which are less efficient for large datasets.

NP-Array vs. Lists

4. Memory Efficiency

- **NumPy Array:** NumPy arrays use less memory compared to lists, especially when dealing with large datasets of the same data type. This is because NumPy arrays store elements in contiguous memory blocks.
- **List:** Lists use more memory because they store references to the objects, which can be of different types and sizes.

5. Convenience and Readability

- **NumPy Array:** For numerical and scientific computing, NumPy arrays provide a more convenient and readable way to perform operations. The syntax for array operations is more concise and clearer.
- **List:** For general-purpose programming, lists are more versatile and simpler to use when the data types are mixed or when the dataset is small.

6. Indexing and Slicing

- **NumPy Array:** NumPy supports advanced indexing and slicing. You can perform operations like slicing multi-dimensional arrays or using boolean arrays for indexing.
- **List:** Lists support basic slicing, but advanced slicing operations and multi-dimensional indexing are more cumbersome.

NP-Array vs. Lists

In Conclusion:

Use **lists** when you need a general-purpose, flexible container that can hold different data types.

Use **NumPy arrays** when you need to perform numerical computations and work with large datasets of the same data type for better performance and efficiency.

Advantages of Using NumPy

- **Performance:** Optimized for speed; much faster than Python lists for numerical operations.
- **Memory Efficiency:** Uses less memory for large datasets.
- **Convenience:** Simplifies complex mathematical operations on arrays.
- **Interoperability:** Compatible with a variety of other libraries and tools (e.g., SciPy, **Pandas**, **Matplotlib**)

Demo

